Deterministic Verification and a Single-Repair Guardrail for LLM-Generated Network Configurations: A Paired, Shared-Candidate Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a sealed, preregistered paired experiment that measured the marginal verifier-detected effect of interposing a deterministic validator plus at most one automated repair on the same LLM-generated configuration candidate. Under the preregistered shared_initial_candidate semantics (exactly one initial generation call per intent; that same candidate reused for both baseline and guarded episodes) we generated one candidate per natural-language intent with gpt-5-mini for N = 250 pairs and evaluated two episodes per intent: baseline (accept candidate) and guarded (deterministic validator; if failing, permit one automated repair). Execution completed 500 episodes with model_calls_used = 251. Results: baseline verifier-pass 249/250 (99.6%), guarded verifier-pass 250/250 (100.0%); contingency n11 = 249, n10 = 1, n01 = 0, n00 = 0; paired difference (guarded - baseline) = +0.4 percentage points, 95% CI [0.0, 1.2 pp] (paired bootstrap percentile, 10,000 resamples, seed = 1729), exact McNemar two-sided p = 1.0. The single discordant pair was converted to pass by the single permitted repair. All execution and analysis artifacts are archived in the registered evidence bundle referenced in the Disclosure Statement.**

I. Introduction

Generative large language models (LLMs) are being explored as intent-to-configuration translators in network operations (NetOps), promising faster authoring of device CLI sequences and intent-driven automation. Stochastic generation, however, creates operational risk: syntax errors, incorrect command ordering, local-policy violations, and vendor-CLI subtleties can produce unsafe or non-functional device changes. Prior empirical and survey work documents LLM feasibility for draft configurations while highlighting the continuing need for deterministic verification or human oversight [1], [2].

A widely adopted architectural response is to interpose a deterministic validator between generation and acceptance, optionally followed by bounded automated repair — a generate–verify–repair (GVR) pattern — which accepts only validator-passed candidates or attempts limited repair when mechanically verifiable defects are detected [3], [4]. These pipelines reduce mechanically verifiable error classes (syntax, ordering, and local-policy violations), but open questions remain about the marginal benefit of such guardrails when modern LLM outputs already conform to validator encodings under constrained task regimes.

This paper reports a preregistered, artifact-backed paired experiment that isolates the marginal verifier+single-repair effect when the same initial candidate is reused across conditions (shared_initial_candidate semantics). By design the experiment measures only the validator-and-repair marginal effect on identical model outputs rather than conflating generation variability across conditions. We executed N_pairs = 250 constrained intent tasks in a synthetic/emulation partition and report concrete, artifact-grounded quantification of the marginal verifier-detected improvement under these conditions. We emphasize that the estimand measured is the paired_success_rate_difference_guarded_minus_baseline on identical initial candidates (no independent per-condition first-pass generation).

II. Related Work

Prior NetOps and LLM evaluation literature motivates GVR architectures and highlights limits of LLM-only correction. Empirical studies of LLMs for network configuration generation report that modern LLMs can produce usable draft configurations under controlled conditions, but typically require deterministic checks or human oversight to address syntax and policy failures [1], [2], [C1]. Deterministic validators and model-checking tools (e.g., Batfish-like analyses, header-space approaches) are established for checking forwarding, reachability, and many local-policy invariants and are natural validators in GVR pipelines [4], [5], [C3].

Work on generate-and-verify and automated repair in related code and configuration domains shows that verifier-guided repair can increase detection and correction of mechanically checkable defects, while remaining limited for deeper semantic correctness that requires richer external tests or operational validation [6], [C2]. Surveys and critical analyses further document that LLM-only self-correction is often unreliable without dependable external feedback, and that prompt-injection and retrieval-based adversaries are practical threats to LLM-integrated workflows [7], [C4], [C5].

Our study differs from most prior work in three respects that follow directly from the preregistration and archived execution: (i) we executed a confirmatory paired design with shared_initial_candidate semantics so both conditions start from the identical generated candidate and any condition difference arises solely from deterministic validation and the single permitted repair; (ii) we report deterministic reconciliation and provider-call accounting that attribute exactly 250 shared initial-generation calls plus 1 repair call (model_calls_used = 251); and (iii) we archive complete validator traces, repair traces, and reconciliation artifacts so contingency counts and

per-stage call accounting are auditable from the evidence bundle.

III. Methodology

Preregistration, design, and estimand. The study was preregistered and specified a confirmatory paired design with N_pairs = 250, generation_semantics = shared_initial_candidate, and the primary estimand paired_success_rate_difference_guarded_minus_baseline.
The preregistration froze the single-repair policy, exclusion and contamination rules, analysis tests (exact McNemar two-sided; paired-bootstrap percentile CIs), and a maximum model-call budget. The preregistration artifact is archived in the evidence bundle referenced in the Disclosure Statement.

Shared_initial_candidate semantics (confirmed). Consistent with the preregistration, for each intent exactly one initial LLM generation was executed and that identical initial candidate was evaluated under both conditions: baseline (the candidate is accepted as-is) and guarded (the candidate is submitted to a deterministic validator; if invalid, allow at most one automated repair and re-validate). This semantics isolates only the marginal effect of deterministic validation and the single permitted repair on the same generation output. Provider-call reconciliation records hosted model call events = 251 (250 shared initial-generation calls + 1 repair call); these counts are reproduced in the archived deterministic_reconciliation artifact.

Task generation and constraints. A deterministic task generator produced 250 constrained intent tasks. Each task payload encoded initial_state, expected_final_state, required_sequence, allowed_touched_fields, and a difficulty label (medium/hard/very_hard). Tasks exercised workflow constraints (make-before-break uplink switchover, paired-link atomic MTU changes, safe access transfer patterns) and enforced protected interfaces that must not be modified. Representative task payloads, required_sequences, and the full task manifest are archived in the evidence bundle; representative validator trace fields (parsed_command_count, required_command_count, final_state snapshots) are included in per-episode scoring artifacts.

Guarded pipeline, validator, and repair. The guarded pipeline used a deterministic network validator (deterministic_netops_validator_v5) that parses CLI sequences, computes parsed_command_count, checks required_sequence ordering, validates workflow constraints, and compares the resulting final_state snapshot to the task's expected_final_state. Validator outputs include normalized_configuration, parsed_command_count, required_command_count, observed_touched_field_count, violation_codes, and a boolean valid flag. Repair templates were deterministic and constrained to syntactic and ordering fixes (reordering commands to match required_sequence, splitting multi-command lines, correcting token-ordering errors). By design no multi-iteration repair or human escalation was permitted; at most one repair call could be executed per episode.

Inclusion and contamination rules. The preregistered exclusion rules removed items that deterministically prevented verification (vendor-parser failures) or contained credential-like tokens or denylist hits. Contamination detectors flagged instruction-like embedded tokens or private-address artifacts. In this confirmatory run no episodes were excluded or flagged: complete_pair_count = 250 and contamination_summary shows zero flagged episodes.

Analysis executor and inference. The preregistered primary analysis used the paired_binary_analysis_v1 executor. The primary test is the exact (two-sided) McNemar on paired binary outcomes; we also computed a paired-bootstrap percentile 95% CI for the guarded - baseline paired difference with 10,000 resamples (bootstrap_seed = 1729). The analysis artifacts and deterministic contingency counts are archived; the key numeric outputs used in inference are reported in Results.

IV. Results

Execution and reconciliation. Execution completed the preregistered confirmatory partition with planned_episode_count = 500, completed_episode_count = 500, complete_pair_count = 250, and model_calls_used = 251. The deterministic reconciliation artifact records hosted_model_call_event_count = 251 with stage_counts: shared_initial_generation = 250 and repair = 1.

Primary binary outcomes and contingency. Validator-scored successes were baseline_success_count = 249 (baseline_success_rate = 0.996) and guarded_success_count = 250 (guarded_success_rate = 1.0). The observed paired contingency is n11 = 249, n10 = 1, n01 = 0, n00 = 0. The single discordant pair (baseline failed, guarded passed) corresponds to the sole exercised automated repair; the observed repair-conversion rate in the confirmatory partition is 1/250 = 0.004 (0.4%). Representative per-episode validator traces for passed episodes show parsed_command_count matching required_command_count for task payloads (examples archived in per-task scoring artifacts).

Statistical inference and uncertainty. The exact McNemar two-sided test on discordant counts (n10 = 1, n01 = 0) returned p = 1.0. The paired-bootstrap percentile 95% confidence interval for the guarded - baseline paired difference (10,000 resamples; bootstrap_seed = 1729) is [0.0, 0.012] (i.e., 0 – 1.2 percentage points). These numeric analysis outputs and bootstrap parameters are recorded in the analysis results artifact.

Representative diagnostics and repair exercise. Validator traces record validation_before and validation_after snapshots, normalized configurations, parsed_command_count, required_command_count, and violation lists (empty for passing episodes). Only one repair was generated and applied; its repair trace and the guarded scoring artifact for that pair are archived and show the repaired candidate satisfied the same deterministic validation checks that the baseline candidate failed. No other repairs were needed in the confirmatory partition.

Missingness and exclusions. No confirmatory episodes were excluded: complete_pairs = 250 and all missingness counts = 0. Contamination detectors flagged no episodes in the confirmatory run.

V. Discussion

Interpreting the near-null marginal verifier effect. Under the preregistered shared_initial_candidate semantics and the constrained synthetic/emulation partition used here, the marginal verifier-detected benefit of inserting a deterministic validator plus a single automated repair was small: initial gpt-5-mini candidates satisfied the deterministic validator in 249/250 cases and the guarded pipeline converted the single failing candidate via the permitted repair. The paired-bootstrap CI upper bound (1.2 percentage points) provides an empirical bound on the maximum verifier-detected improvement under this pipeline and task scope.

Why the observed guarded gain is small (artifact-grounded factors). Three artifact-grounded factors from the preregistration and execution explain the small observed gain: (1) the deterministic task generator encoded constrained intents, allowed_touched_fields, and required_sequences that reduced ambiguity and favored well-formed CLI outputs; (2) the declared model (gpt-5-mini) produced highly consistent, syntactically well-formed sequences under these constrained prompts; and (3) the deterministic validator checks mechanically verifiable properties (syntax, required_sequence, final_state snapshot) and does not capture broader behavioral correctness in live protocol execution, cross-device emergent effects, or vendor-implementation nuances outside the validator model.

Power, baseline rates, and practical interpretation. The preregistration power calculation assumed a baseline pass rate $\approx$ 0.90 when selecting N_pairs = 250 to target detection of a 15 percentage-point absolute effect with $\approx$80% power. The realized baseline pass rate (0.996) is substantially higher than assumed; this reduces the chance of observing a statistically significant paired difference for small absolute improvements. Thus the practical output of the confirmatory run is the CI bound on the maximum verifier-detected improvement for this task scope rather than a rejection of a large-effect null hypothesis.

Operational and design implications. These results do not imply validators are unnecessary in general. Instead they quantify the marginal verifier-detected benefit under specific, constrained conditions: (a) in tightly constrained emulation-style tasks and with a capable LLM, the incremental verifier-detected improvement from a single automated repair may be small; (b) validators still provide deterministic guarantees on mechanically checkable properties and are essential when task scope, vendor heterogeneity, operational stakes, or adversarial inputs broaden; (c) more expressive validators, multi-iteration repair policies, human escalation, or external dynamic tests (e.g., live reachability checks, forwarding-state verification) could materially change operational benefits; and (d) production heterogeneity and adaptive adversaries remain open risks that were not exercised here.

Observed failure modes, diagnostics, and auditability. The confirmatory artifacts reveal the dominant failure mode here was ordering/tokensequence mismatches that a deterministic, template-driven repair could fix in a single pass; other failure classes (vendor-specific semantic mismatches, cross-device emergent errors) were not observed in this controlled task bank. All per-episode validator traces, repair traces, and reconciliation artifacts are archived to support audit and follow-up analyses.

Limitations and external validity. The confirmatory scope is limited to the synthetic/emulation partition and a single declared model and validator implementation; preregistered community and production-like partitions were not executed in this run. The deterministic validator assessed mechanically verifiable properties but not live protocol-level correctness. Adversarial prompt-injection, retrieval-augmented generation attacks, and large-scale production heterogeneity were not evaluated. These limitations restrict generalization and motivate broader follow-up studies.

VI. Limitations

- Confirmatory scope limited to the synthetic/emulation partition; preregistered community and production-like partitions were not executed and remain exploratory.
- Deterministic validator assessed mechanically verifiable properties (syntax and prescribed workflow sequences) but did not measure broader semantic or behavioral correctness (protocol dynamics, emergent interactions).
- Single declared model (gpt-5-mini) and one validator implementation were used; results do not establish multi-model or alternate-validator generality.
- Only one automated repair attempt was permitted by design; multi-iteration repairs or human-in-the-loop escalation were not evaluated.
- Adversarial and retrieval-based attacks (adaptive prompt-injection, vendor-targeted exploit vectors) and large-scale production heterogeneity were not exercised and remain open threats.

VII. Conclusion

We executed a sealed, preregistered paired experiment under shared_initial_candidate semantics to measure the verifier-detected marginal benefit of deterministic validation plus one permitted automated repair on identical LLM-generated configuration candidates. In N = 250 constrained synthetic intent tasks, initial gpt-5-mini candidates passed the deterministic validator in 249/250 cases; the guarded pipeline converted the single failing candidate via the single permitted repair. Observed paired improvement = 0.4 percentage points (95% CI [0.0, 1.2 pp]; p = 1.0). These results bound the verifier-detected marginal improvement for the studied pipeline and task scope but do not settle broader operational, adversarial, multi-model, or multi-iteration repair questions. All execution and analysis manifests, raw responses, validator traces, repair

traces, and analysis artifacts are archived and available in the evidence bundle described in the Disclosure Statement.

Disclosure Statement

Human role and autonomy boundary: After an autonomy lock required by the CNSM Agentic AI Researcher track, the autonomous system performed literature search, study selection, preregistration freeze, prompt preservation, model calls, deterministic validation, permitted repair execution, analysis, drafting, peer-review, revision, and finalization without manual human editing of technical content. A human Research Director selected the broad topic family and performed pre-lock infrastructure development only; no human manually edited manuscript technical content after the autonomy lock.

Models, tools, and framework: declared model = gpt-5-mini; deterministic validator = deterministic_netops_validator_v5; deterministic task generator = deterministic_netops_task_generator_v5; adapter family = hosted_netops_gvr_v1. Execution used a guarded generate–verify–repair flow permitting at most one automated repair per episode. Provider-call accounting reconciles to model_calls_used = 251 (250 shared initial_generation + 1 repair) with per-stage counts recorded in the archived reconciliation artifact.

Preregistration and immutable prompt reference: the study was preregistered under the stated study identifier and the immutable master prompt used to initiate the autonomous run is archived as provenance/master_prompt.txt with SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196 add77f5acdf1582bb39b15ba. The evidence bundle contains the archived preregistration, execution manifests, raw responses, validator traces, repair traces, and analysis artifacts referenced in this manuscript.

References

[1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296

[2] Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, Rui Zhang, "When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs", 2024, 10.1162/tacl_a_00713

[3] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194

[4] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866

[5] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899

[6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, Mario Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", 2023, 10.1145/3605764.3623985

[7] http://arxiv.org/abs/2403.09369